Why a Tree Model Beat a Neural Network at Breast Cancer Subtype Classification

A controlled comparison of XGBoost, 1D-CNN, and ensembles on TCGA-BRCA PAM50 prediction

Final Project Report — Artificial Intelligence in Genomics

Vishvaksen Pendyala (vp2781) — May 2026

1. Background

Cancers from different organs look different under a microscope, but they can also be told apart by their molecular activity — in particular, which genes are switched on or off inside their cells. Modern sequencing technology measures the activity of approximately 20,000 genes simultaneously, producing a gene expression profile for each patient sample. Two tumors that look the same under a microscope can have very different gene expression profiles, and those molecular differences often predict which treatment a patient will respond to.

Breast cancer is a useful test case because it has a well-established molecular classification called PAM50. PAM50 assigns every breast tumor to one of five subtypes — Luminal A, Luminal B, Basal-like, HER2-enriched, or Normal-like — based on the expression of fifty selected genes. Each subtype responds differently to hormonal therapy, chemotherapy, and targeted drugs, suggesting the need for subtype clarification during clinical validation.

Commented [MK1]: informal

Predicting subtypes from gene expression data is hard because the data is wide: each patient has around 20,000 features, but only a few hundred patients are typically available for any given cancer type. This is the opposite of the situation most modern machine learning was designed for, where there are far more training examples than features. Two model families handle this situation in different ways: gradient-boosted decision trees (such as XGBoost) treat each gene independently and select the most informative ones, while one-dimensional convolutional networks (1D-CNNs) slide small filters across the gene vector and detect patterns formed by groups of nearby genes.

2. Motivation

This project asks two questions. First, can a model trained only on gene expression recover the PAM50 subtype labels without being given the original fifty PAM50 genes as a shortcut? Second, do XGBoost and a 1D-CNN make different kinds of mistakes, so that combining them in an ensemble outperforms either model alone? The first question tests whether the broader collection of all RNA molecules carries enough signal to recover the clinical categories. The second question tests whether tree-based and neural models are complementary using a controlled experiment. This is a common methodological claim, but as it turns out, the experiment refutes it for this task and provides a clear biological explanation for why.

3. Methods

3.1 Data

We used the TCGA breast invasive carcinoma cohort downloaded from UCSC Xena. The expression matrix contained log2-normalized RSEM(RNA-Seq by Expectiation-Maximization) counts for 20,531 genes across 1,218 tumor samples sequenced on the Illumina HiSeq platform. The phenotype file provided RNA-seq–derived PAM50 calls for 956 samples; the remaining samples had no subtype call and were excluded. The 956 samples were distributed across all five subtypes: Luminal A (434), Luminal B (194), Basal-like (142), Normal-like (119), and HER2-enriched (67). No class was below a 10-sample threshold so none were dropped.

I filtered out genes with variance below 0.01 across all samples, removing about 680 essentially silent genes and leaving 19,847 features. Features were then z-score standardized using the training set statistics. Labels were integer-encoded. The data was split 70/15/15 into training (690 samples), validation (122), and held-out test (144) sets, stratified by subtype, so that each split preserved the class proportions.

3.2 Models

I trained two model families and built two ensembles.

XGBoost was the first model — a gradient-boosted tree ensemble. The input is the full 19,847-dimensional standardized expression vector, and the output is a probability distribution over the five subtypes. I searched 60 hyperparameter combinations covering tree depth (4–8), learning rate (0.01–0.2), number of trees (300–1000), subsampling rates, and L1 / L2 regularization. Each configuration was trained with early stopping on the validation set. GPU acceleration via XGBoost's native CUDA support reduced per-trial wall time from approximately 5 minutes on the CPU to under 10 seconds on an A100 GPU.

The second model was a 1D-CNN implemented using PyTorch. It treats the gene expression vector as a sequence and slides convolutional filters across it. The architecture has a wide stem convolution (kernel size 15, stride 2) for initial downsampling, followed by residual convolutional blocks, then global average pooling, and a fully connected classifier. To control overfitting on 690 training samples, I used dropout of 0.3–0.6 at every block, label smoothing, weight decay, and early stopping on validation loss. I searched 10 hyperparameter combinations covering filter count (64–256), kernel size (5–11), depth (2–4 blocks), and dropout rate.

Because an initial CNN sweep performed poorly, I ran a second sweep on a version of the input in which genes were reordered by chromosomal position (so that genes physically adjacent on the genome were adjacent in the input vector) and reduced to the top 2,000 by variance. This was designed to test whether the CNN's underperformance was due to the input lacking biological spatial structure. Both CNN versions used the same architecture search space.

Finally, two ensembles combined the best XGBoost model with the best CNN. Soft voting takes a weighted average of the two probability vectors with the weight α optimized on the validation set. Stacking concatenates the two probability vectors and feeds them into a small logistic regression meta-learner trained on validation predictions.

3.3 HPC Workflow

All hyperparameter searches were run on the NYU HPC Burst cluster using a single NVIDIA A100 GPU. XGBoost trials were run sequentially because each trial saturated the device; the full 60-trial sweep was completed in approximately 10 minutes. CNN trials likewise ran sequentially, each taking roughly one minute, with the full 10-trial sweep finishing in under fifteen minutes. The same workload on CPU would have taken several hours, an order-of-magnitude difference that made the controlled comparison feasible within the project timeline.

4. Results

4.1 Baseline

A Random Forest baseline with 100 trees reached 83.6% validation accuracy. The per-class F1 scores already revealed the task's difficulty structure: Basal-like was easily separated (F1 = 0.95), HER2-enriched was difficult (F1 = 0.50, with only 9 validation samples), and Luminal A versus Luminal B showed the classical biological confusion (F1 = 0.86 and 0.73, respectively). This confirmed that PAM50 subtyping is a non-trivial task with meaningful headroom for improvement; unlike a naive five-cancer-type classification on the same dataset family, which reaches 100% accuracy at the same baseline and removes the case for hyperparameter optimization entirely.

4.2 XGBoost hyperparameter sweep

Across the 60 XGBoost trials, validation accuracy ranged from 84.4% to 91.0%, showing that the choice of hyperparameters mattered. The best configuration used a learning rate of 0.10, maximum tree depth of 4, and 300 estimators — moderate values throughout. Two of the top three configurations used shallow trees (depth 4), suggesting that depth-4 trees regularize well on this small, wide dataset, whereas deeper trees begin to overfit. Figure 1 shows the distribution of validation accuracy across the sweep.

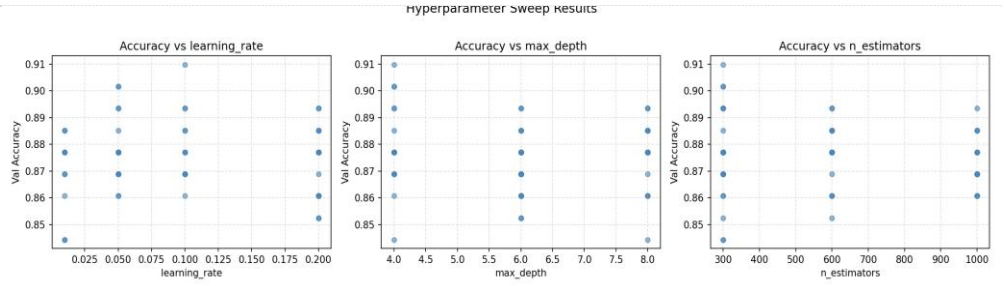


Figure 1. Validation accuracy across all 60 XGBoost trials, plotted against learning rate, tree depth, and number of estimators. The best configurations cluster at a moderate learning rate (0.05–0.10) and shallow trees (depth 4).

4.3 Test-set comparison

The best XGBoost model was retrained on the combined training and validation sets and evaluated on the held-out test set, which was never used during hyperparameter search. Table 1 reports the final test-set metrics for all four model variants.

Model	Test Accuracy	F1 (macro)
Random Forest (baseline)	~84%	0.79
XGBoost (best trial)	**90.28%**	**0.878**
1D-CNN (best trial, ordered)	62.50%	0.533
Soft-Voting Ensemble	90.28%	0.878
Stacking Ensemble	89.58%	0.870

Table 1. Test-set accuracy and macro F1 for the baseline, the two individual models, and the two ensembles. The test set (N = 144) was held out from all hyperparameter searches and not touched until the end.

XGBoost achieved 90.28% test accuracy and a macro F1 of 0.878, beating the Random Forest baseline by approximately 6 percentage points. The CNN achieved 62.50% test accuracy, below the baseline. The ensembles did not improve over XGBoost alone: soft voting reproduced the XGBoost result exactly because the validation-set

optimization assigned α = 1.0 (zero weight to the CNN), and stacking actually performed slightly worse (89.58%) because the meta-learner could not fully ignore the noisy CNN signal. Figure 2 visualizes this comparison.

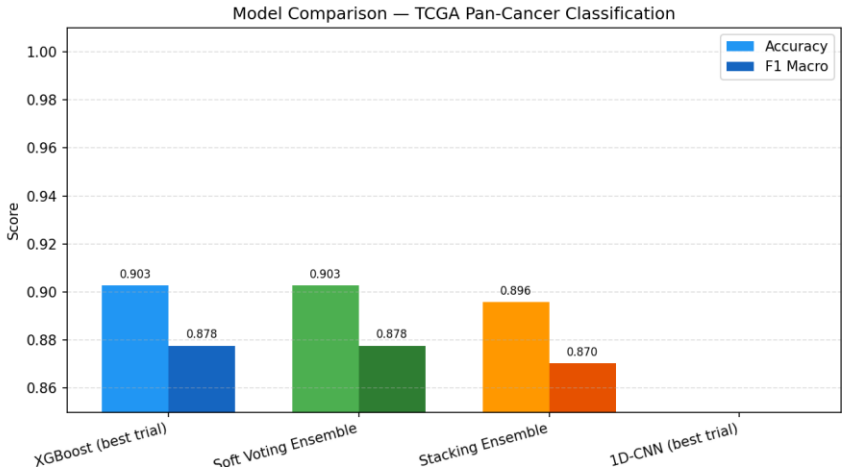


Figure 2. Test-set accuracy and macro F1 across all four models. XGBoost and the soft-voting ensemble are identical because the validation-optimized ensemble weight gave the CNN zero weight.

4.4 Where the CNN failed

Figure 3 breaks down per-class F1 scores. The CNN failed completely on the HER2-enriched class (F1 = 0), the smallest subtype with only nine test samples — it never predicted HER2 for any test sample. It performed reasonably on the dominant Luminal A class (F1 ≈ 0.82) but poorly on Luminal B (F1 ≈ 0.37). XGBoost in contrast handled HER2 well (F1 = 0.76) and held up on Luminal B (F1 = 0.85). The CNN's failure mode is consistent with the model collapsing toward the dominant class under class imbalance — a known risk when training neural networks on small, imbalanced data.

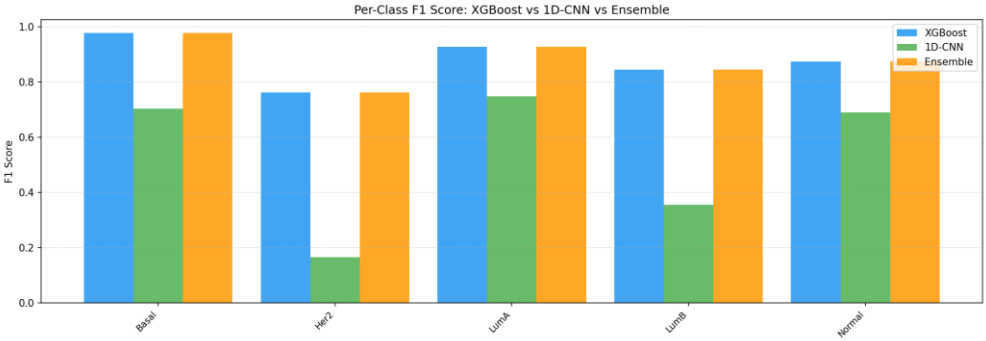


Figure 3. Per-class F1 score for XGBoost, the 1D-CNN, and the soft-voting ensemble. The CNN failed to predict any HER2 sample (F1 = 0) and was poor on the transcriptionally similar Luminal B class. The ensemble matched XGBoost class by class.

4.5 The chromosomal-reordering experiment

Our initial hypothesis for the CNN's underperformance was that the input had no biological spatial structure — gene columns were ordered arbitrarily (alphabetically by HUGO symbol), so the convolutional filters were sliding over a meaningless arrangement. I tested this by downloading gene-position annotations from UCSC Xena, sorting the 19,847 genes by chromosomal position, selecting the top 2,000 by variance (preserving genomic order), and retraining the CNN sweep on this reordered input. The hypothesis was that genes physically adjacent on the same chromosome would tend to co-regulate (because of shared regulatory elements and topologically-associated domains), so the CNN's local kernels would now have meaningful patterns to detect.

The hypothesis failed. The reordered CNN's validation accuracy range (60.7–71.3%) was statistically indistinguishable from the original arbitrary-order CNN (62.3–71.3%), and the best reordered model reached only 62.5% test accuracy. This null result tells us something useful about PAM50 biology: the discriminative signal lives in individual marker genes (ESR1 on chromosome 6, ERBB2 on 17, MKI67 on 10), not in local patterns of co-expression between chromosomal neighbors. Putting marker genes in chromosomal order does not create useful spatial structure because the relevant markers are scattered across the genome rather than clustered.

4.6 Biomarker discovery

XGBoost's feature importance score functions as a candidate biomarker ranking. Figure 4 shows the top 30 genes by importance. The list rediscovers the established biology of breast cancer subtyping: FOXM1 (the top-ranked gene) is the master proliferation transcription factor that distinguishes the aggressive Luminal B subtype from the quiescent Luminal A; GATA3 is a luminal lineage transcription factor in the original PAM50 panel; MLPH (melanophilin), TFF3, and ANXA9 are well-established luminal markers; STARD3 sits in the HER2 amplicon on chromosome 17q12, immediately adjacent to ERBB2 itself; EGFR is a basal-like marker; and MYH11 is a smooth-muscle gene elevated in Normal-like tissue. The model independently rediscovered the two major molecular axes of breast cancer (proliferation and luminal-versus-basal lineage) that PAM50 was originally designed around — a strong sanity check that it learned biologically meaningful features rather than spurious correlations.

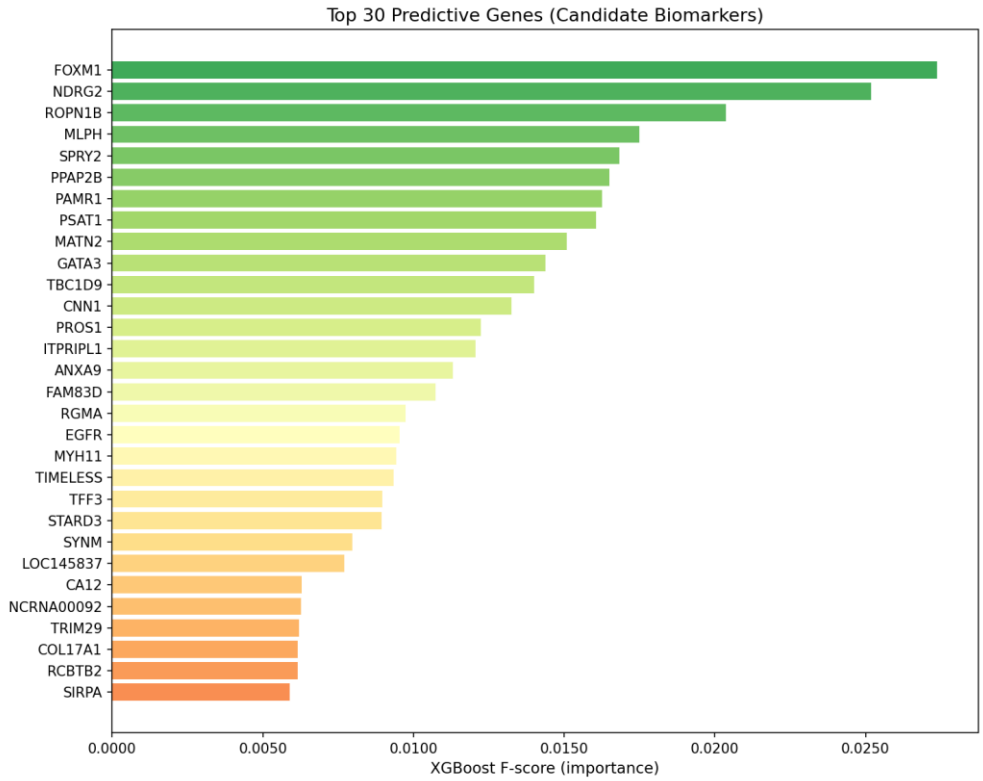


Figure 4. Top 30 genes ranked by XGBoost feature importance. The list independently rediscovers genes from the original PAM50 panel (FOXM1, GATA3) and from neighboring subtype-defining loci (STARD3 next to ERBB2).

4.7 Data structure

To visualize the structure that the models had to learn, I projected the training set into two dimensions with UMAP (Figure 5). Normal-like samples form an isolated cluster, consistent with their distinct gene expression program (Normal samples are largely tissue-adjacent biopsies, not tumors). Basal-like samples form a second distinct cluster, explaining the classifier's near-perfect performance on this class (22 of 22 correct). Luminal A, Luminal B, and HER2 overlap substantially in the main cloud — this overlap is exactly where the residual classification errors occur.

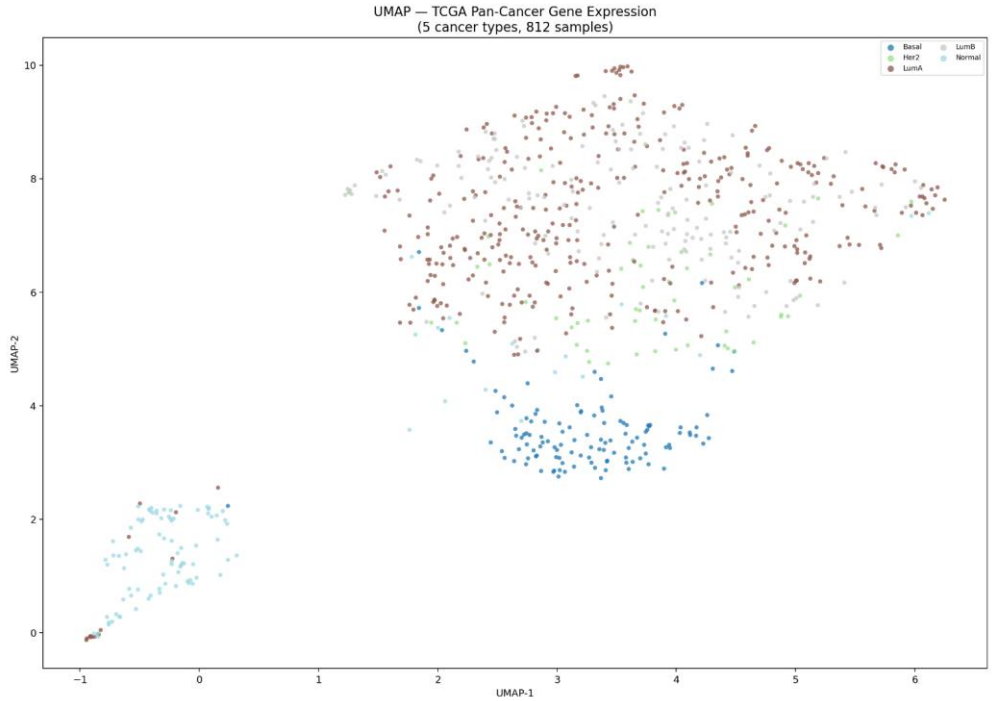


Figure 5. UMAP projection of the training set, colored by PAM50 subtype. Normal-like and Basal-like form isolated clusters and are classified perfectly; Luminal A, Luminal B, and HER2 overlap and account for most residual error.

5. Discussion and Conclusion

Two findings stand out. First, an XGBoost model trained on raw gene expression — with no hint about which genes are diagnostic — recovers PAM50 subtypes with 90.3% test accuracy, roughly six percentage points above the Random Forest baseline and within the practical range of clinical PAM50 assays. The top-ranked features in the trained model match the established biology, which is the strongest sanity check that the high accuracy reflects real subtype signal rather than dataset artifacts.

Second, the 1D-CNN failed at this task and the failure was reproducible. The original hypothesis — that a CNN would complement XGBoost by capturing local co-expression patterns between adjacent genes — was directly falsified by the chromosomal-reordering experiment, in which giving the CNN a biologically structured input did not change its performance. Soft voting between the two models found $\alpha = 1.0$ (zero weight for the CNN), and stacking actually performed slightly worse than XGBoost alone. The methodological lesson is concrete: ensembling does not help when one component has no usable signal, and a 1D-CNN over arbitrary or even chromosomally-ordered genes does not have usable signal for marker-gene-based classification tasks like PAM50.

Three limitations are worth flagging. (1) Sample size: we had 956 labeled samples across five subtypes, which is at the low end for a CNN. A transfer-learning approach starting from a model pretrained on a much larger pan-cancer cohort would likely do better and is the obvious follow-up. (2) Single cohort: all data come from TCGA, and TCGA-trained models are known to generalize unevenly to other sequencing centers because of batch effects. External validation on METABRIC or other independent BRCA cohorts is the natural next step. (3) Architectural breadth:

we only tried 1D-CNNs as the neural counterpart. A different neural architecture — for example, a graph neural network operating on a protein-protein interaction graph or a transformer attending over genes — would have access to non-local structure that a 1D-CNN cannot reach, and would be a more rigorous test of the "neural networks help on tabular biomedical data" claim.

The most useful single takeaway is that on a wide, small, marker-driven biomedical classification task, a well-tuned gradient-boosting tree model is hard to beat, and the standard recipe of "add a deep network and ensemble" can actively hurt performance if the deep network has no spatial or sequential structure to exploit. HPC-parallel hyperparameter sweeps remain valuable in this setting — not because they enable bigger models, but because they enable careful comparison and the kind of controlled architecture experiment (such as the chromosomal-reordering test) that would otherwise be too time-consuming to perform within a project timeline.

References

Goldman, M. J. et al. (2020). Visualizing and interpreting cancer genomics data via the Xena platform. Nature Biotechnology 38, 675–678.

Parker, J. S. et al. (2009). Supervised risk predictor of breast cancer based on intrinsic subtypes. Journal of Clinical Oncology 27, 1160–1167.

Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16, 785–794.

McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection. arXiv:1802.03426.

Cancer Genome Atlas Network. (2012). Comprehensive molecular portraits of human breast tumours. Nature 490, 61–70.

Appendix A: Code and Data

All source code, data preprocessing scripts, and analysis scripts are included in the supplementary ZIP. The pipeline consists of (1) data acquisition and preprocessing, (2) XGBoost hyperparameter grid generation, (3) a per-trial XGBoost training script, (4) results analysis and plotting, (5) a per-trial CNN training script, (6) CNN hyperparameter grid generation, (7) a chromosomal-reordering preprocessing script, and (8) the ensemble analysis. A README in the supplementary archive describes the run order. The TCGA expression and clinical data used in this work is publicly available from UCSC Xena (https://xenabrowser.net) under TCGA Research Network terms of use.